COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 Data Structures And Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	09/02/26	**DATE SUBMITTED:**	09/02/26
NAME:	Acala, Matt John C.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HOA 6.1 Searching Techniques

1. PROCEDURE OUTPUT

Linear Search Array

Header

searchingAlgo.h > ...

```cpp
#include "node.h"

#ifndef SEARCHINGALGO_H
#define SEARCHINGALGO_H

template<typename T>
void linearSearchAR(int size, T dataFind, T array[]) {

    for (int j = 0; j < size; j++) {
        if(dataFind == array[j]) {
            std::cout << "Searching is successful\n";
            return;
        }

    }
    std::cout << "Searching is unsuccessful\n";
}

```

Main

main-6.cpp > main()

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "searchingAlgo.h"

const int max_size = 50;

int main() {

    int dataset[max_size];

    srand(time(0));

    for (int i = 0; i < max_size; i++) {
        dataset[i] = rand();
        std::cout << dataset[i] << " ";
    }

    std::cout << std::endl;

    std::cout << "What number to find: ";
    int choice;

    std::cin >> choice;

    linearSearchAR(max_size, choice, dataset);

    return 0;
}
```

Output

```
PS C:\Programming\C++\SCHOOL\CPE 010\HOA 6p1> cd "c:\Programming\C++\SCHOOL\CPE 010\HOA 6p1\" ; if ($?) { g++ main-6.cpp -o main-6 } ; if ($?) { .\main-6 }
23954 1573 4757 31424 31231 29807 32329 4140 15196 2479 6079 2148 5156 3401 27740 20617 15620 5381 15196 26935 14641 18447 15555 25007 29951 21314 21654 3638 1
2976 29349 1610 32184 14729 13366 15485 29361 31532 22503 14564 25422 8993 28903 16517 17615 17590 11298 22566 9775 6921 24587
What number to find: 6079
Searching is successful
PS C:\Programming\C++\SCHOOL\CPE 010\HOA 6p1> cd "c:\Programming\C++\SCHOOL\CPE 010\HOA 6p1\" ; if ($?) { g++ main-6.cpp -o main-6 } ; if ($?) { .\main-6 }
24059 17842 19354 15018 849 24390 855 8772 19279 2349 1346 3558 25472 8596 16994 18379 21827 5164 8243 26061 19489 22317 4510 20099 5726 14747 15399 6572 5070
26312 32105 20432 1770 17178 25791 19748 31779 1550 30021 19375 15223 13693 17407 22699 30785 26142 18007 14638 13141 25933
What number to find: 1
Searching is unsuccessful
PS C:\Programming\C++\SCHOOL\CPE 010\HOA 6p1>
```

Observation

The program accurately scans the randomly generated dataset for the user's input. For instance, searching for a missing number like 1 results in an "unsuccessful" message. Conversely, when searching for a number that was

actually generated in the array, such as 6079, the program successfully locates it and outputs a "successful" message.

Linear Search (Linked List)

Header

```
template<typename T>
void linearSearchLL(Node<T> *nodes, T dataFind) {

while(nodes != nullptr){

    if(nodes->data == dataFind){
        std::cout << "The data: " << nodes->data << " was found successfully.\n";
        return;
        }
    nodes = nodes->next;

    }
    std::cout << "Searching is unsuccessful.\n";
}
```

Main

```
    Node<char> *char1 = newNode('M');
    Node<char> *char2 = newNode('A');
    Node<char> *char3 = newNode('T');
    Node<char> *char4 = newNode('T');

    char1->next = char2;
    char2->next = char3;
    char3->next = char4;

    // Searching for a letter that exists in the list
    linearSearchLL(char1, 'T');

    // Searching for a letter that does not exist in the list
    linearSearchLL(char1, 'S');

    return 0;
}
```

Output

```
The data: T was found successfully.
Searching is unsuccessful.
```

Observation

When we call the linearSearchLL(char1, 'T') function, the program walks through the list node by node until it hits the third one, which holds the first 'T'. That's why it prints "The data: T was found successfully." Since 'S' doesn't exist in any of the nodes, the search reaches the end of the list without a match, so it prints "Searching is unsuccessful."

Binary Search Array

Header

```
template<typename T>
void binarysearchArr(T data[], int arrSize, T dataFind){
    int low = 0, high = arrSize - 1, middle;
    while(low <= high){
        middle = (int)((low + high / 2));
        if(dataFind == data[middle]){
            std::cout << "Data was found.\n";
            return;
        }
        else if(dataFind < data[middle]){
            high = middle - 1;

        }
        else{
            low = middle + 1;
        }
    }
    std::cout << "Data was not found\n";


}
```

Main

```
int Arr1[10] = {1,2,3,4,5,6,7,8,9,10};

for(int i = 0; i < max_size; i++) {
    Arr1[i] = i;
}

binarysearchArr(Arr1, 10, 11);

int Arr2[10] = {1,2,3,4,5,6,7,8,9,10};

for(int i = 0; i < max_size; i++) {
    Arr2[i] = i;
}

binarysearchArr(Arr2, 10, 1);

return 0;
}
```

Output

```
Data was not found
Data was found.
```

Observation

When searched for 11, the program printed "Data was not found" because the array only held values from 1-10 after the loop, so 11 simply wasn't there. But when searched for 1, the program printed "Data was found" since 1 exists in that same range.

Binary Search Linked List

Header

```cpp
template <typename T>
Node<T>* getMiddle(Node<T> *start, Node<T>* last){

    Node<T>* slow = start;
    Node<T>* fast = start;

    //continue until fast reaches the end or the last node of the current search range
    while(fast != last && fast->next != last) {

        //move slow one step
        slow = slow->next;
        //move fast two steps
        fast = fast->next->next;
    }

    return slow;
}

template <typename T>
void binarySearchLL(Node<T>* start, T dataFind){
    //set start to the head of the list and set last to null
    Node<T>* last = nullptr;

    while(start != last){

        Node<T> *middle = getMiddle(start, last);

        // If the middle element is same as the key to be searched, we return it.
        if(middle->data == dataFind){
            std::cout << "Searching is successful\n";
            return;
        }
        // Else if middle element is greater than the key to be searched,
        // we have to search in the left side of the singly linked list.
        else if(middle->data > dataFind){
            last = middle;
        }
        // Else if middle element is less than the key to be searched,
        // we have to search in the right side of the singly linked list.
        else{
            start = middle->next;
        }
    }

    std::cout << "Searching is unsuccessful\n";
}
```

Main

```
Node<char> *char1 = newNode('M');
Node<char> *char2 = newNode('A');
Node<char> *char3 = newNode('T');
Node<char> *char4 = newNode('T');

char1->next = char2;
char2->next = char3;
char3->next = char4;
char4->next = nullptr;

binarySearchLL(char1, 'T');
binarySearchLL(char1, 'M');
```

Output

```
Searching is successful
Searching is unsuccessful
```

Observation

When searched for 'T', the program prints "Searching is successful" because the algorithm's calculated middle node happens to land right on it. But when you search for 'M', even though it's sitting at the head of the list, the binary search logic skips over it since the list isn't sorted alphabetically. That's why the output shows "Searching is unsuccessful".

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1. Suppose you are doing a sequential search of the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Utilizing both a linked list and an array approach to the list, use sequential search and identify how many comparisons would be necessary to find the key ‘18’?

Code:

```
int dataset[10] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
int target = 18;


// ============Problem 1============
// ====Sequential Search Array=====
linearSearchAR(10, target, dataset);

// =====Sequential Search Linked List=====
Node<int> *head = newNode(dataset[0]);
Node<int> *current = head;
for (int i = 1; i < 10; i++) {
    current->next = newNode(dataset[i]);
    current = current->next;
}

linearSearchLL(head, target);
```

Output:

```
Searching is successful
The data: 18 was found successfully.
```

Observation: Both the array and linked list approaches require exactly two comparisons to find the target. This is because the sequential search algorithm checks each element one by one, starting from the very beginning of the list. Since the target 18 is located at the second position, the search finds it immediately and stops, without needing to check for the other 18 that appears later in the dataset.

Problem 2. Modify your sequential search algorithm so that it returns the count of repeating instances for a given search element ‘k’. Test on the same list given in problem 1

Code:

```
template<typename T>
int countSequentialAR(int size, T dataFind, T array[]) {
    int count = 0;
    for (int j = 0; j < size; j++) {
        if (dataFind == array[j]) {
            count++;
        }
    }
    return count;
}

template<typename T>
int countSequentialLL(Node<T> *nodes, T dataFind) {
    int count = 0;
    Node<T> *current = nodes;
    while (current != nullptr) {
        if (current->data == dataFind) {
            count++;
        }
        current = current->next;
    }
    return count;
}
```

```
int arcount = countSequentialAR(10, target, dataset);
int llcount = countSequentialLL(head, target);
std::cout << "Array Count for key '" << target << "': " << arcount << std::endl;
std::cout << "Linked List Count for key '" << target << "': " << llcount  << std::endl;
```

Output:

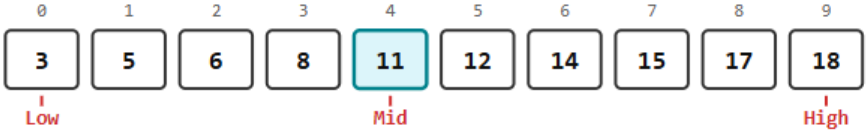
```
Array Count for key '18': 2
Linked List Count for key '18': 2
```

Observation: array and linked list algorithms detect 2 occurrences of the target key ‘18’ at index 1 and index 4

Problem 3. Suppose you have the following sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] and are using the binary search algorithm. If you wanted to find the key 8, draw a diagram that shows how the searching works per iteration of the algorithm. Prove that your drawing is correct by implementing the algorithm and showing a screenshot of the code and the output console.

Iteration 1: Initial Search Space

Target = 8. Calculate Midpoint: (0 + 9) / 2 = 4.



Observation: 8 < 11 (Mid). Target must be in the left half.
Action: Update High = Mid - 1 (Index 3).

Iteration 2: Narrowed Search Space

Target = 8. Calculate Midpoint: (0 + 3) / 2 = 1.



Observation: 8 > 5 (Mid). Target must be in the right half.
Action: Update Low = Mid + 1 (Index 2).

Iteration 3: Further Narrowing

Target = 8. Calculate Midpoint: (2 + 3) / 2 = 2.



Observation: 8 > 6 (Mid). Target must be in the right half.
Action: Update Low = Mid + 1 (Index 3).

Iteration 4: Match Found

Target = 8. Calculate Midpoint: (3 + 3) / 2 = 3.



Code:

```cpp
template<typename T>
void binSearchArr(T data[], int arrSize, T dataFind) {
    int low = 0, high = arrSize - 1, middle;
    int i= 1;
    while (low <= high) {
        middle = (int)((low + high / 2));
        std::cout << "Iteration " << i << ": low=" << low << " [" << data[low]
                  << "], high=" << high << " [" << data[high]
                  << "], middle=" << middle << " [" << data[middle] << "]\n";

        if (dataFind == data[middle]) {
            std::cout << "Key " << dataFind << " successfully found at index " << middle << ".\n";
            return;
        } else if (dataFind < data[middle]) {
            high = middle - 1;
        } else {
            low = middle + 1;
        }
        i++;
    }
    std::cout << "Key " << dataFind << " was not found.\n";
}
```

Output:

```
Iteration 1: low=0 [3], high=9 [18], middle=4 [11]
Iteration 2: low=0 [3], high=3 [8], middle=1 [5]
Iteration 3: low=2 [6], high=3 [8], middle=3 [8]
Key 8 successfully found at index 3.
```

Observation: It works by checking the middle and slicing up the search to either the left or right side up until the item is found

Problem 4. Modify the binary search algorithm so that the algorithm becomes recursive. Using this new recursive binary search, implement a solution to the same problem for problem 3.

Code:

```cpp
template<typename T>
int binarySearchRecursiveAR(T array[], int low, int high, T dataFind, int depth = 1) {
    // Base Case 1: Target not in list
    if (low > high) {
        std::cout << "Key " << dataFind << " was not found.\n";
        return -1;
    }

    int middle = low + (high - low) / 2;
    std::cout << "Recursion Call " << depth << ": low=" << low
              << " [" << array[low] << "], high=" << high
              << " [" << array[high] << "], middle=" << middle
              << " [" << array[middle] << "]\n";

    // Base Case 2: Target found
    if (dataFind == array[middle]) {
        std::cout << "Key " << dataFind << " successfully found at index " << middle << ".\n";
        return middle;
    }
    // Recursive Step: Search left half
    else if (dataFind < array[middle]) {
        return binarySearchRecursiveAR(array, low, middle - 1, dataFind, depth + 1);
    }
    // Recursive Step: Search right half
    else {
        return binarySearchRecursiveAR(array, middle + 1, high, dataFind, depth + 1);
    }
}
```

```cpp
binarySearchRecursiveAR(sortedData, 0, 9, target1);
```

Output:

```
Recursion Call 1: low=0 [3], high=9 [18], middle=4 [11]
Recursion Call 2: low=0 [3], high=3 [8], middle=1 [5]
Recursion Call 3: low=2 [6], high=3 [8], middle=2 [6]
Recursion Call 4: low=3 [8], high=3 [8], middle=3 [8]
Key 8 successfully found at index 3.
```

Observation: Same as problem 3 it calls itself with a smaller part of the list until the target is found.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Concept Explanator, Example Generator, and Analogy Generator**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of

concepts, etc.

Extent of Use: **Since this is a new topic some of is too overwhelming therefore i need AI to slice it up for me**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.